PENGANTAR Konsep Pengembangan PL

OLEH :

TIM DOSEN KPPL

Tujuan Perkuliahan

Memberikan gambaran tentang perangkat lunak, proses pengembangan perangkat lunak.

Memberikan gambaran tentang cakupan pengembangan perangkat lunak, pentingnya RPL dan siapa-siapa saja yang terlibat di dalamnya.

Agenda

Definisi Software

Macam-macam software

Mitos seputar software

Definisi Software Engineering (RPL)

Mengapa perlu SE

Bagaimana seharusnya diterapkan SE

Kapan diterapkan SE

Siapa yang terlibat SE

Apa atribut perangkat lunak yang baik?

Definisi *Software* (Perangkat Lunak)

Perangkat lunak adalah sekumpulan item atau objek yang membentuk konfigurasi yang
mencakup

- Program
- Dokumen
- Data

Definisi *Software* (Perangkat Lunak)

Program komputer, prosedur, dan kemungkinan terkait dokumentasi dan data yang terkait dengan pengoperasian dari sistem komputer

(IEEE Standard* Glossary of Software Engineering Terminology, 1990*)

Definisi *Software* (Perangkat Lunak)

Software dirancang dan dibangun oleh Insinyur perangkat lunak.

Software digunakan oleh hampir semua orang dalam masyarakat

Insinyur Perangkat Lunak memiliki kewajiban moral untuk membangun software yang handal sehingga tidak akan merugikan orang lain.

Pengguna Perangkat Lunak hanya peduli apakah produk perangkat lunak telah memenuhi harapan mereka dan membuat tugas-tugas mereka lebih mudah untuk diselesaikan.

Definisi *Software* (Perangkat Lunak)

Software merupakan sebuah produk dan kendaraan untuk memberikan suatu produk (informasi).

Perangkat lunak direkayasa tidak diproduksi.

Perangkat lunak tidak rusak, tetapi memburuk.

Definisi *Software* (Perangkat Lunak)

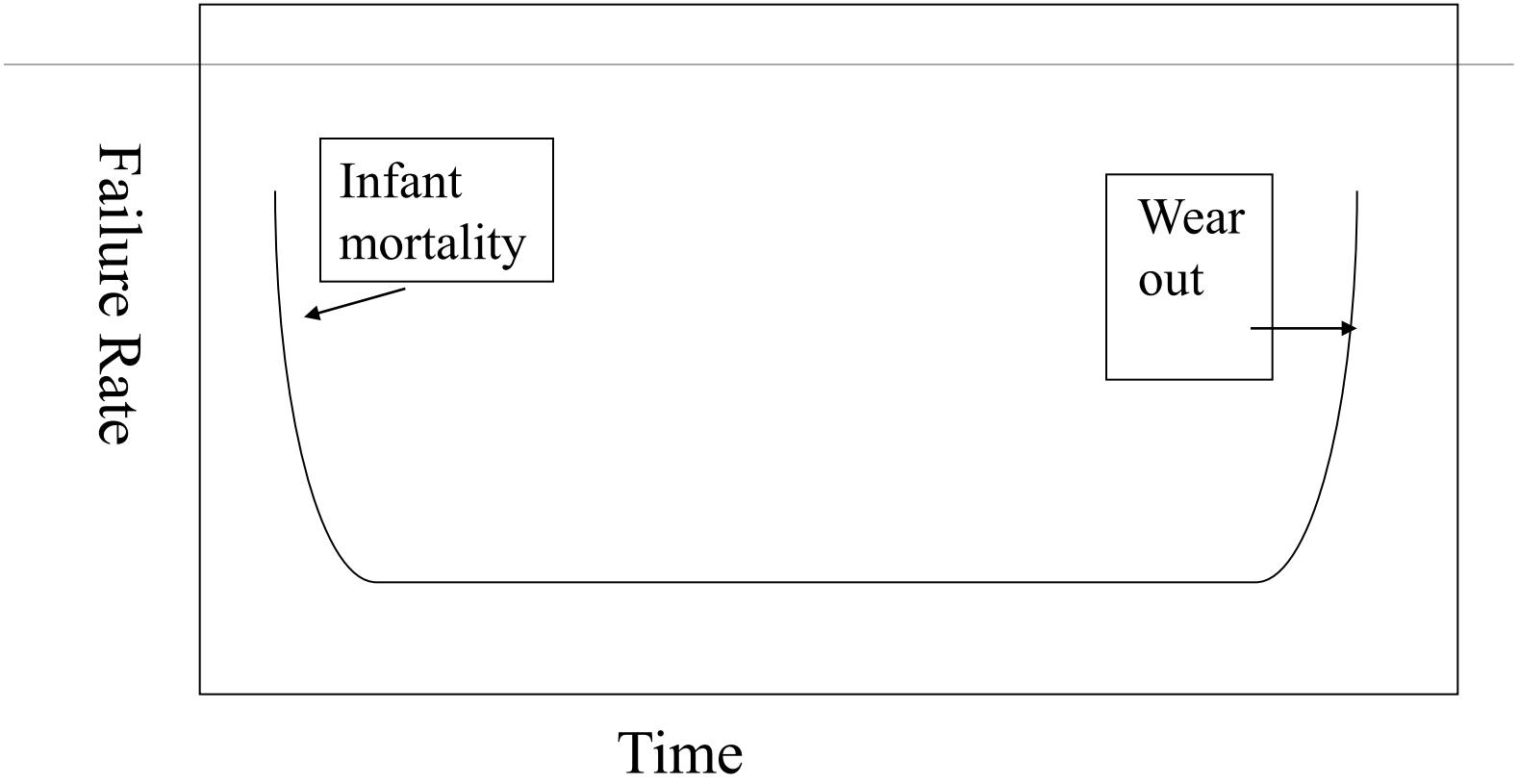
Software merupakan sebuah produk dan kendaraan untuk memberikan suatu produk (informasi).

Perangkat lunak direkayasa tidak diproduksi.

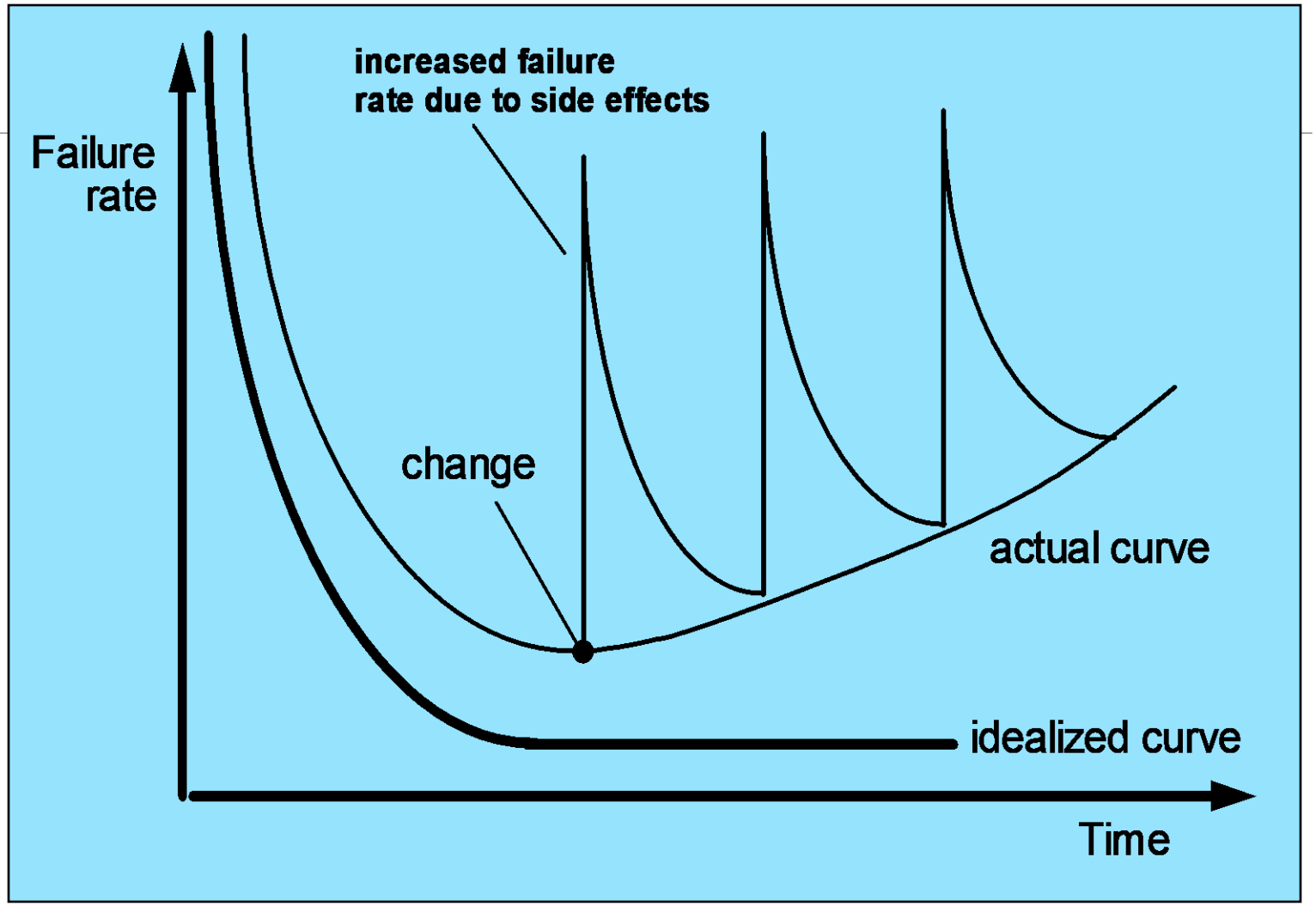
Perangkat lunak tidak rusak, tetapi memburuk.

Saat ini, sebagian besar perangkat lunak dibangun secara custom (dikembangkan untuk pelanggan secara individual sesuai spesifikasi mereka)

Failure (“Bathtub”) Curve for Hardware



Wear vs. Deterioration for SoftWare



Jenis-Jenis Aplikasi Perangkat Lunak

System software

Application software

Embedded software

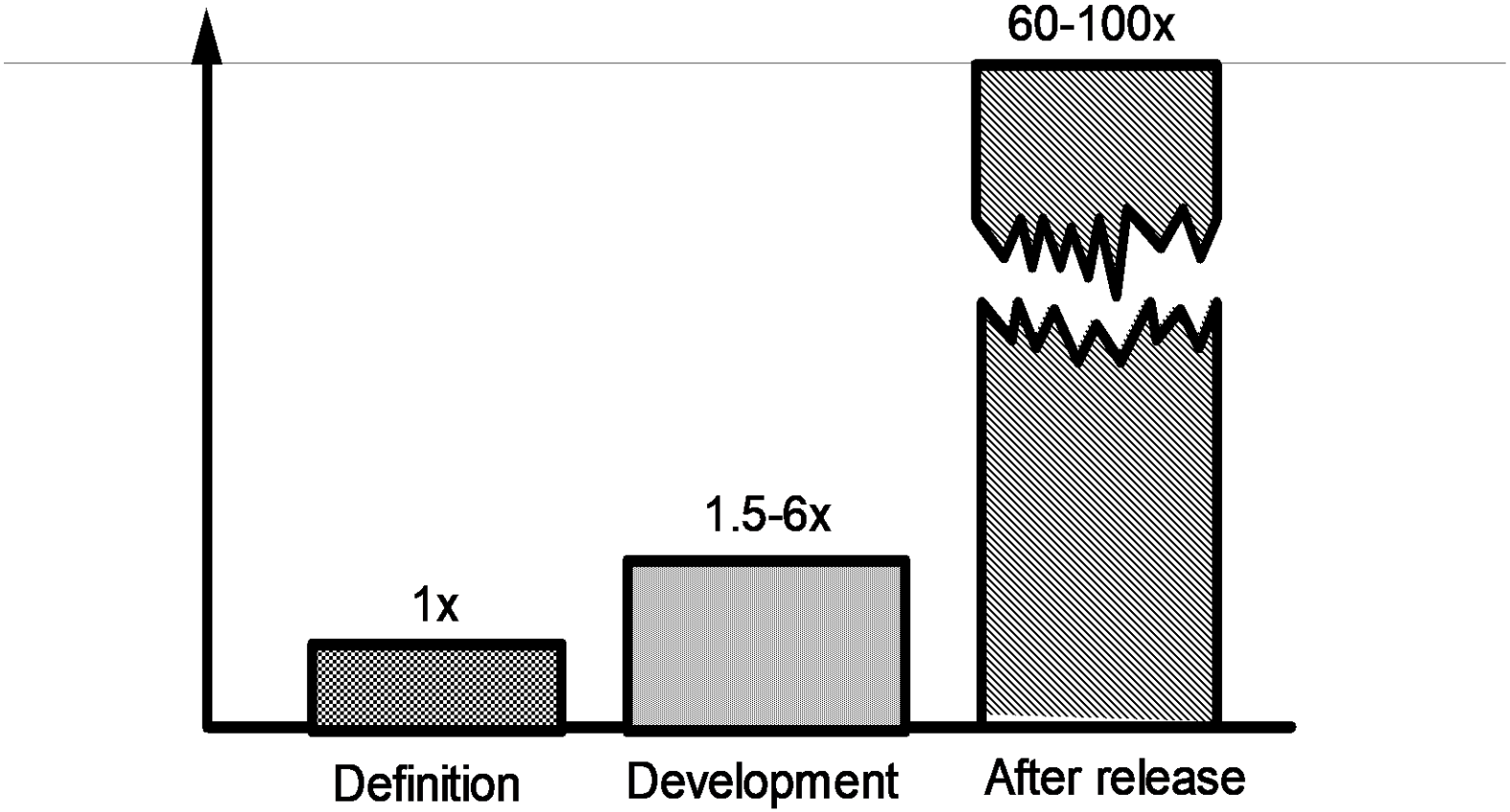
Engineering/Scientific software

Product software

Web Applications

Artificial intelligence software

The Cost of Change



Mitos Software

Masih diyakini oleh banyak manajer dan praktisi

Membahayakan karena mereka memiliki unsur-unsur kebenaran

Setiap praktisi dan manajer harus memahami realitas bisnis perangkat lunak

Mitos Software:
Dari sudut pandang Client

MITOS	REALITA
Sebuah pernyataan umum dari tujuan cukup untuk mendapatkan selanjutnya. Isian secara rinci dilakukan kemudian.	Definisi yang tidak rinci diawal requirement atau kebutuhan adalah penyebab utama kegagalan dan keterlambatan perangkat lunak.
Kebutuhan Proyek terus berubah, tetapi perubahan dapat dengan mudah diakomodasi karena software fleksibel.	Biaya dari perubahan untuk perangkat lunak untuk memperbaiki suatu kesalahan meningkat secara dramatis pada tahap selanjutnya dari kehidupan software.

Mitos Software:
Dari sudut pandang Developer

MITOS	REALITA
Setelah program ditulis dan bekerja, pekerjaan pengembang telah selesai.	50% -70% dari usaha pengeluaran program terjadi setelah dikirim ke pelanggan.
Sampai program berjalan, tidak ada cara untuk menilai kualitas.	Review Software dapat lebih efektif dalam menemukan kesalahan daripada pengujian untuk kelas-kelas tertentu yang memiliki kesalahan.
Hanya diserahkan untuk proyek yang sukses adalah program yang jalan.	Suatu konfigurasi perangkat lunak meliputi dokumentasi, regenerasi file, masukan (input) data uji, dan hasil data uji.

Mitos Software:
Dari sudut pandang Manajemen

MITOS	REALITA
Ada Buku standar sehingga perangkat lunak yang dikembangkan akan memuaskan.	Buku mungkin ada, tetapi mereka biasanya tidak up to date dan tidak digunakan.
Komputer dan kakas (tools) perangkat lunak yang tersedia sudah cukup.	CASE tools diperlukan tetapi biasanya tidak diperoleh atau tidak digunakan.
Kita selalu dapat menambahkan jumlah programmer jika proyek terlambat penyelesaiannya	"Menambahkan orang untuk proyek yang terlambat akan membuat penyelesaian perangkat lunak lebih terlambat "-. Brooks

Apa Software Engineering (Rekayasa Perangkat Lunak)

Rekayasa Perangkat Lunak adalah pembentukan dan prinsip-prinsip rekayasa yang tepat untuk mendapatkan perangkat lunak yang ekonomis, dapat dipercaya dan bekerja efisien pada mesin sesungguhnya (Fritz Bauer, 1969)

Rekayasa Perangkat Lunak adalah [1] penerapan secara sistematis, disiplin, pendekatan terukur terhadap pembangunan, operasi, dan pemeliharaan
perangkat lunak, dan [2] studi tentang pendekatan seperti pada [1]
(IEEE, 1993)

Mengapa RPL (1)

Untuk membuat perangkat lunak dengan benar (proses) dan mendapatkan perangkat lunak yang benar (produk)

Adanya Kompleksitas Perangkat Lunak

- *Domain problem: Business Rule*
- *Ukuran Data : Digital and Non Digital*
- *Solution: Algorithm*
- *Tempat atau Situs*

Mengapa RPL (2)

Perangkat Lunak harus benar

Kebenaran perangkat lunak harus bisa dipelihara

Bagaimana seharusnya diterapkan RPL

Cakupan RPL (2 hal)

- Produk = Software
  - Programs
  - Documents
  - Data
- Proses bagaimana membangun perangkat lunak
  - Management process
  - Technical process

Produk RPL

Produk diperoleh melalui tahapan Pengembangan = Software Development Life

Cycle (SDLC)

Contoh siklus hidup (SDLC):

- Waterfall model
- V model
- Spiral model
- Fountain model
- Prototyping
- Agile

Proses RPL

Management Process meliputi:

- Project management
- Configuration management
- Quality Assurance management

Proses RPL

Technical Process, digambarkan sebagai metode yang akan diterapkan dalam tahap tertentu dari SDLC

- Analysis methods
- Design methods
- Programming methods
- Testing methods

Metode teknis ini yang memunculkan paradigma seperti berorientasi terstruktur, objek, aspek, dll

Kapan diterapkan RPL

Pre-project

Project Initiation

Project Realisation

Software Delivery & Maintenance

Siapa yang terlibat RPL

Manager
- Project Manager
- Configuration Manager
- Quality Assurance Manager

Software Developer:
- Analyst
- Designer
- Programmer

Support
- Administration
- Technical Support for Customer (help desk, customer care)
- Welfare (Kesejahteraan)

CASE (Computer-Aided Software Engineering)

Perangkat lunak sistem yang dimaksudkan untuk memberikan dukungan otomatis untuk kegiatan proses perangkat lunak. CASE sistem sering digunakan untuk mendukung metode.

- Upper-CASE
  - Tools (Alat/Kakas) untuk mendukung proses kegiatan awal penggalian kebutuhan, analisis dan desain
- Lower-CASE
  - Alat untuk mendukung kegiatan berikutnya seperti pemrograman, debugging dan pengujian.

Apa atribut perangkat lunak yang baik

Perangkat lunak ini harus memberikan fungsi yang diperlukan dan kinerja bagi pengguna dan harus dipertahankan, diandalkan dan dapat diterima.

- Maintainability (mudah dipelihara/dirawat)
  - Software harus berevolusi untuk memenuhi perubahan kebutuhan
- Dependability (dapat diandalkan)
  - Software harus dapat dipercaya
- Efficiency (Daya Guna)
  - Efisien dalam penggunaan sumber daya sistem (memori, hardware, listrik, dll)
- Acceptability (Dapat diterima)
  - Perangkat Lunak harus diterima oleh pengguna seperti yang pernah dirancang. Ini berarti harus bisa dimengerti, digunakan dan kompatibel dengan sistem lainnya.

Apa saja tantangan utama yang dihadapi RPL?

Heterogeneity (keberagaman)

- Mengembangkan teknik untuk membangun perangkat lunak yang dapat mengatasi perbedaan platform dan lingkungan eksekusi.

Delivery (pengiriman)

- Mengembangkan teknik yang mengakibatkan pengiriman perangkat lunak lebih cepat.

Trust (kepercayaan)

- Mengembangkan teknik yang menunjukkan bahwa perangkat lunak dapat dipercaya oleh para penggunanya.

Pustaka

Pressman, R. S., *Software Engineering: A Practitioner's Approach*, 8th Edition, McGraw-Hill, 2008, chapter 1

Sommerville, I., Software Engineering 8th edition, Addison-Wesley, 2007, chapter 1